

```

/* USER CODE BEGIN Header */
/**

*****
****
* @file           : main.c
* @brief          : Main program body

*****
****
* @attention
*
* Copyright (c) 2026 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in the
LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*

*****
****
*/
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "cmsis_os.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

```

```

/* Private variables -----
-----*/
COM_InitTypeDef BspCOMInit;
ADC_HandleTypeDef hadc1;
ADC_HandleTypeDef hadc2;
DMA_HandleTypeDef hdma_adc1;
DMA_HandleTypeDef hdma_adc2;

DAC_HandleTypeDef hdac1;

I2C_HandleTypeDef hi2c1;

TIM_HandleTypeDef htim1;
TIM_HandleTypeDef htim2;

osThreadId defaultTaskHandle;
osThreadId PulseSensorTaskHandle;
osThreadId ThermalSensorTaHandle;
osMessageQId PulseSensorDataQueueHandle;
osMessageQId ThermalSensorDataQueueHandle;
/* USER CODE BEGIN PV */

/* Temperature Sensor */
#define ADC_RESOLUTION_THERMAL 4095
#define V_REF_THERMAL 3.3
#define TEMP_SENSITIVITY 0.01
#define NUM_SAMPLES 10
float voltage_thermal;
float temperature;
float adc_value_thermal;
float Average_Temp;

/* Pulse Sensor */
#define ibm_threshold 60000
const float ALPHA = 0.1;
float adc_value_pulse;
float raw;
float heart_beat_value;
float Average_Pulse;
float last_beat_time = 0;
float current_beat_time = 0;
float time_between_beats = 0;
float beat_detected = 0;
int initialised = 0;
int chosen_pulse = 0;
int max = 9;
int min = 0;
/*bpms*/
int Pulse_Scared [] = {90, 98, 102, 104, 109, 110, 112, 118, 126, 130};
int Pulse_Excited [] = {85, 80, 79, 76, 72, 68, 66, 63, 60, 58};

/*Counts*/
int TrackA_Count = 0;
int TrackB_Count = 0;
int Default_Count = 0;
int Random_Count = 0;

/*Thresholds*/

```

```

int Temp_Threshold = 28;
int Pulse_Threshold = 89;

/*Switch case*/
int state;
int deciding_done = 0;
int run = 0;

/* Private variables */
uint8_t RX_Buffer [1] ;
int emotion;
volatile uint8_t      gFlagPendBus;
volatile uint8_t      gFlagRecvBus;
volatile uint8_t      gFlagErrrBus;

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
void PeriphCommonClock_Config(void);
static void MPU_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_ADC1_Init(void);
static void MX_ADC2_Init(void);
static void MX_DAC1_Init(void);
static void MX_TIM1_Init(void);
static void MX_I2C1_Init(void);
static void MX_TIM2_Init(void);
void StartDefaultTask(void const * argument);
void PulseSensorTaskHandler(void const * argument);
void ThermalSensorTaskHandler(void const * argument);

/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
}

void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
    if (hi2c->Instance == I2C1)
    {
        state = RX_Buffer[0];

        // immediately re-arm FIRST (important)
        HAL_I2C_Slave_Receive_IT(&hi2c1, RX_Buffer, 1);
    }
}

```

```

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MPU Configuration-----*/
    MPU_Config();

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the
    SysTick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* Configure the peripherals common clocks */
    PeriphCommonClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_ADC1_Init();
    MX_ADC2_Init();
    MX_DAC1_Init();
    MX_TIM1_Init();
    MX_I2C1_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */
    HAL_TIM_Base_Start_IT(&htim1);

    /* Enable IRQs */
    HAL_NVIC_EnableIRQ(I2C1_ER_IRQn);
    HAL_NVIC_EnableIRQ(I2C1_EV_IRQn);
    HAL_NVIC_EnableIRQ(TIM6_DAC_IRQn);

    HAL_I2C_Slave_Receive_IT(&hi2c1, RX_Buffer, 1);
    HAL_Delay(100);

```

```

emotion = RX_Buffer[0];
/* USER CODE END 2 */

/* USER CODE BEGIN RTOS_MUTEX */
/* add mutexes, ... */
/* USER CODE END RTOS_MUTEX */

/* USER CODE BEGIN RTOS_SEMAPHORES */
/* add semaphores, ... */
/* USER CODE END RTOS_SEMAPHORES */

/* USER CODE BEGIN RTOS_TIMERS */
/* start timers, add new ones, ... */
/* USER CODE END RTOS_TIMERS */

/* Create the queue(s) */
/* definition and creation of PulseSensorDataQueue */
osMessageQDef(PulseSensorDataQueue, 10, uint16_t);
PulseSensorDataQueueHandle =
osMessageCreate(osMessageQ(PulseSensorDataQueue), NULL);

/* definition and creation of ThermalSensorDataQueue */
osMessageQDef(ThermalSensorDataQueue, 10, uint16_t);
ThermalSensorDataQueueHandle =
osMessageCreate(osMessageQ(ThermalSensorDataQueue), NULL);

/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
/* USER CODE END RTOS_QUEUES */

/* Create the thread(s) */
/* definition and creation of defaultTask */
osThreadDef(defaultTask, StartDefaultTask, osPriorityNormal, 0, 128);
defaultTaskHandle = osThreadCreate(osThread(defaultTask), NULL);

/* definition and creation of PulseSensorTask */
osThreadDef(PulseSensorTask, PulseSensorTaskHandler, osPriorityIdle, 0,
128);
PulseSensorTaskHandle = osThreadCreate(osThread(PulseSensorTask),
NULL);

/* definition and creation of ThermalSensorTa */
osThreadDef(ThermalSensorTa, ThermalSensorTaskHandler, osPriorityIdle,
0, 128);
ThermalSensorTaHandle = osThreadCreate(osThread(ThermalSensorTa),
NULL);

/* USER CODE BEGIN RTOS_THREADS */
/* add threads, ... */
/* USER CODE END RTOS_THREADS */

/* Initialize leds */
BSP_LED_Init(LED_GREEN);
BSP_LED_Init(LED_YELLOW);
BSP_LED_Init(LED_RED);

/* Initialize USER push-button, will be used to trigger an interrupt
each time it's pressed.*/

```

```

BSP_PB_Init(BUTTON_USER, BUTTON_MODE_EXTI);

/* Initialize COM1 port (115200, 8 bits (7-bit data + 1 stop bit), no
parity */
BspCOMInit.BaudRate    = 115200;
BspCOMInit.WordLength  = COM_WORDLENGTH_8B;
BspCOMInit.StopBits    = COM_STOPBITS_1;
BspCOMInit.Parity      = COM_PARITY_NONE;
BspCOMInit.HwFlowCtl    = COM_HWCONTROL_NONE;
if (BSP_COM_Init(COM1, &BspCOMInit) != BSP_ERROR_NONE)
{
    Error_Handler();
}

/* Start scheduler */
osKernelStart();

/* We should never get here as control is now taken by the scheduler */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /*AXI clock gating */
    RCC->CKGAENR = 0xE003FFFF;

    /** Supply configuration update enable
    */
    HAL_PWREx_ConfigSupply(PWR_DIRECT_SMPS_SUPPLY);

    /** Configure the main internal regulator output voltage
    */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE0);

    while(!__HAL_PWR_GET_FLAG(PWR_FLAG_VOSRDY)) {}

    /** Initializes the RCC Oscillators according to the specified
    parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType =
    RCC_OSCILLATORTYPE_HSI|RCC_OSCILLATORTYPE_LSI;

```

```

RCC_OscInitStruct.HSISState = RCC_HSI_DIV1;
RCC_OscInitStruct.HSICalibrationValue = 64;
RCC_OscInitStruct.LSISState = RCC_LSI_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
RCC_OscInitStruct.PLL.PLLM = 4;
RCC_OscInitStruct.PLL.PLLN = 9;
RCC_OscInitStruct.PLL.PLLP = 2;
RCC_OscInitStruct.PLL.PLLQ = 4;
RCC_OscInitStruct.PLL.PLLR = 2;
RCC_OscInitStruct.PLL.PLLRGE = RCC_PLL1VCIRANGE_3;
RCC_OscInitStruct.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
RCC_OscInitStruct.PLL.PLLFRACN = 0;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
*/
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2
|RCC_CLOCKTYPE_D3PCLK1|RCC_CLOCKTYPE_D1PCLK1;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.SYSCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.AHBCLKDivider = RCC_HCLK_DIV1;
RCC_ClkInitStruct.APB3CLKDivider = RCC_APB3_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_APB1_DIV1;
RCC_ClkInitStruct.APB2CLKDivider = RCC_APB2_DIV1;
RCC_ClkInitStruct.APB4CLKDivider = RCC_APB4_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief Peripherals Common Clock Configuration
 * @retval None
 */
void PeriphCommonClock_Config(void)
{
    RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};

    /** Initializes the peripherals clock
    */
    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_ADC;
    PeriphClkInitStruct.PLL2.PLL2M = 4;
    PeriphClkInitStruct.PLL2.PLL2N = 8;
    PeriphClkInitStruct.PLL2.PLL2P = 3;
    PeriphClkInitStruct.PLL2.PLL2Q = 2;
    PeriphClkInitStruct.PLL2.PLL2R = 2;
    PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_3;
    PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
    PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
    PeriphClkInitStruct.AdcClockSelection = RCC_ADCCLKSOURCE_PLL2;

```

```

    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief ADC1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_ADC1_Init(void)
{

    /* USER CODE BEGIN ADC1_Init 0 */

    /* USER CODE END ADC1_Init 0 */

    ADC_MultiModeTypeDef multimode = {0};
    ADC_ChannelConfTypeDef sConfig = {0};

    /* USER CODE BEGIN ADC1_Init 1 */

    /* USER CODE END ADC1_Init 1 */

    /** Common config
    */
    hadcl.Instance = ADC1;
    hadcl.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV16;
    hadcl.Init.Resolution = ADC_RESOLUTION_10B;
    hadcl.Init.ScanConvMode = ADC_SCAN_DISABLE;
    hadcl.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
    hadcl.Init.LowPowerAutoWait = DISABLE;
    hadcl.Init.ContinuousConvMode = DISABLE;
    hadcl.Init.NbrOfConversion = 1;
    hadcl.Init.DiscontinuousConvMode = DISABLE;
    hadcl.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadcl.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadcl.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DR;
    hadcl.Init.Overrun = ADC_OVR_DATA_PRESERVED;
    hadcl.Init.LeftBitShift = ADC_LEFTBITSHIFT_NONE;
    hadcl.Init.OversamplingMode = DISABLE;
    if (HAL_ADC_Init(&hadcl) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure the ADC multi-mode
    */
    multimode.Mode = ADC_MODE_INDEPENDENT;
    if (HAL_ADCEx_MultiModeConfigChannel(&hadcl, &multimode) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure Regular Channel
    */
    sConfig.Channel = ADC_CHANNEL_5;

```



```

sConfig.Rank = ADC_REGULAR_RANK_1;
sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;
sConfig.SingleDiff = ADC_SINGLE_ENDED;
sConfig.OffsetNumber = ADC_OFFSET_NONE;
sConfig.Offset = 0;
sConfig.OffsetSignedSaturation = DISABLE;
if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN ADC1_Init 2 */

/* USER CODE END ADC1_Init 2 */

}

/**
 * @brief ADC2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_ADC2_Init(void)
{

    /* USER CODE BEGIN ADC2_Init 0 */

    /* USER CODE END ADC2_Init 0 */

    ADC_ChannelConfTypeDef sConfig = {0};

    /* USER CODE BEGIN ADC2_Init 1 */

    /* USER CODE END ADC2_Init 1 */

    /** Common config
    */
    hadc2.Instance = ADC2;
    hadc2.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV16;
    hadc2.Init.Resolution = ADC_RESOLUTION_12B;
    hadc2.Init.ScanConvMode = ADC_SCAN_DISABLE;
    hadc2.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
    hadc2.Init.LowPowerAutoWait = DISABLE;
    hadc2.Init.ContinuousConvMode = DISABLE;
    hadc2.Init.NbrOfConversion = 1;
    hadc2.Init.DiscontinuousConvMode = DISABLE;
    hadc2.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadc2.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadc2.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DR;
    hadc2.Init.Overrun = ADC_OVR_DATA_PRESERVED;
    hadc2.Init.LeftBitShift = ADC_LEFTBITSHIFT_NONE;
    hadc2.Init.OversamplingMode = DISABLE;
    if (HAL_ADC_Init(&hadc2) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure Regular Channel
    */

```

```

sConfig.Channel = ADC_CHANNEL_3;
sConfig.Rank = ADC_REGULAR_RANK_1;
sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;
sConfig.SingleDiff = ADC_SINGLE_ENDED;
sConfig.OffsetNumber = ADC_OFFSET_NONE;
sConfig.Offset = 0;
sConfig.OffsetSignedSaturation = DISABLE;
if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN ADC2_Init 2 */

/* USER CODE END ADC2_Init 2 */

}

/**
 * @brief DAC1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_DAC1_Init(void)
{
    /* USER CODE BEGIN DAC1_Init 0 */

    /* USER CODE END DAC1_Init 0 */

    DAC_ChannelConfTypeDef sConfig = {0};

    /* USER CODE BEGIN DAC1_Init 1 */

    /* USER CODE END DAC1_Init 1 */

    /** DAC Initialization
    */
    hdac1.Instance = DAC1;
    if (HAL_DAC_Init(&hdac1) != HAL_OK)
    {
        Error_Handler();
    }

    /** DAC channel OUT1 config
    */
    sConfig.DAC_SampleAndHold = DAC_SAMPLEANDHOLD_DISABLE;
    sConfig.DAC_Trigger = DAC_TRIGGER_NONE;
    sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE;
    sConfig.DAC_ConnectOnChipPeripheral = DAC_CHIPCONNECT_DISABLE;
    sConfig.DAC_UserTrimming = DAC_TRIMMING_FACTORY;
    if (HAL_DAC_ConfigChannel(&hdac1, &sConfig, DAC_CHANNEL_1) != HAL_OK)
    {
        Error_Handler();
    }

    /** DAC channel OUT2 config
    */
    if (HAL_DAC_ConfigChannel(&hdac1, &sConfig, DAC_CHANNEL_2) != HAL_OK)

```

```

    {
        Error_Handler();
    }
    /* USER CODE BEGIN DAC1_Init 2 */

    /* USER CODE END DAC1_Init 2 */

}

/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2C1_Init(void)
{
    /* USER CODE BEGIN I2C1_Init 0 */

    /* USER CODE END I2C1_Init 0 */

    /* USER CODE BEGIN I2C1_Init 1 */

    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.Timing = 0x10808DD3;
    hi2c1.Init.OwnAddress1 = 36;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;

    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure Analogue filter
    */
    if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) !=
HAL_OK)
    {
        Error_Handler();
    }

    /** Configure Digital filter
    */
    if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN I2C1_Init 2 */

    /* USER CODE END I2C1_Init 2 */

```

```

}

/**
 * @brief TIM1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM1_Init(void)
{
    /* USER CODE BEGIN TIM1_Init 0 */

    /* USER CODE END TIM1_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM1_Init 1 */

    /* USER CODE END TIM1_Init 1 */
    htim1.Instance = TIM1;
    htim1.Init.Prescaler = 999;
    htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim1.Init.Period = 3999;
    htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim1.Init.RepetitionCounter = 0;
    htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterOutputTrigger2 = TIM_TRGO2_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) !=
        HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM1_Init 2 */

    /* USER CODE END TIM1_Init 2 */

}

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{

```

```

/* USER CODE BEGIN TIM2_Init 0 */

/* USER CODE END TIM2_Init 0 */

TIM_ClockConfigTypeDef sClockSourceConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig = {0};

/* USER CODE BEGIN TIM2_Init 1 */

/* USER CODE END TIM2_Init 1 */
htim2.Instance = TIM2;
htim2.Init.Prescaler = 72-1;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 4294967295;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */

/* USER CODE END TIM2_Init 2 */

}

/**
 * Enable DMA controller clock
 */
static void MX_DMA_Init(void)
{
    /* DMA controller clock enable */
    __HAL_RCC_DMA1_CLK_ENABLE();

    /* DMA interrupt init */
    /* DMA1_Stream0_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(DMA1_Stream0_IRQn, 5, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream0_IRQn);
    /* DMA1_Stream1_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 5, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);
}

```

```

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_2|GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6,
GPIO_PIN_RESET);

    /*Configure GPIO pins : PE2 PE4 PE5 PE6 */
    GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

    /*Configure GPIO pins : PA0 PA8 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_8;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /*Configure GPIO pins : PE7 PE8 */
    GPIO_InitStruct.Pin = GPIO_PIN_7|GPIO_PIN_8;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

const uint8_t step_sequence[8][4] =
{
    {1,0,0,0},
    {1,1,0,0},
    {0,1,0,0},
    {0,1,1,0},
    {0,0,1,0},
    {0,0,1,1},

```

```

        {0,0,0,1},
        {1,0,0,1}
    };

void step_motor(int step)
{
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_2, step_sequence[step][0]);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4, step_sequence[step][1]);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, step_sequence[step][2]);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, step_sequence[step][3]);
}

void rotate_angle_forwards(float angle)
{
    int total_cycles = (angle * (4096 / 360.0) / 8.0);
    int total_steps = total_cycles * 8;

    if (run == 0)
    {
        for(int i = 0; i < total_steps; i++)
        {
            int step = i % 8;
            float progress = i / total_steps;
            int delay_time;

            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0)
            {
                step_motor(step);
                osDelay(5);
                EmergencyStop();
            }
            else if (BSP_PB_GetState(BUTTON_USER) == 1)
            {
                OverrideSystem();
            }
            else if (progress < 0.3)
            {
                delay_time = 5 - (progress * 10);
            }
            else if (progress > 0.7)
            {
                delay_time = 2 + ((progress - 0.7) * 10);
            }
            else
            {
                delay_time = 2;
            }

            if (delay_time < 1) delay_time = 1;
            if (delay_time > 10) delay_time = 10;

            run = 1;
            step_motor(step);
            osDelay(delay_time);
        }
    }
}

```

```
}
```

```
void rotate_angle_backwards(float angle)
{
    int total_cycles = (angle * (4096 / 360.0) / 8.0);
    int total_steps = total_cycles * 8;

    if (run == 1)
    {
        for(int i = total_steps; i >= 0; i--)
        {
            int step = i % 8;

            float progress = i / total_steps;

            int delay_time;

            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0)
            {
                step_motor(step);
                osDelay(5);
                EmergencyStop();
            }
            else if (BSP_PB_GetState(BUTTON_USER) == 1)
            {
                OverrideSystem();
            }
            else if (progress < 0.3)
            {
                delay_time = 5 - (progress * 10);
            }
            else if (progress > 0.7)
            {
                delay_time = 2 + ((progress - 0.7) * 10);
            }
            else
            {
                delay_time = 2;
            }

            if (delay_time < 1) delay_time = 1;
            if (delay_time > 10) delay_time = 10;

            run = 2;
            step_motor(step);
            osDelay(delay_time);
        }
    }
}
```

```
void EmergencyStop(void)
{
```



```

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0)
    {
        state = 4;
    }

}

void OverrideSystem (void)
{
    if (BSP_PB_GetState(BUTTON_USER) == 1)
    {
        state = 5;
    }

}

void camera(emotion)
{
    if (emotion == 0)
    {
        Default_Count = Default_Count +1;
    }
    else if (emotion == 1)
    {
        TrackB_Count = TrackB_Count +1;
    }
    else if (emotion == 2)
    {
        TrackA_Count = TrackA_Count +1;
    }
}

void ThermalAndPulseSensor(void)
{
    if (Average_Temp > Temp_Threshold && Average_Pulse >
Pulse_Threshold)
    {
        TrackB_Count = TrackB_Count +1;
    }
    else if (Average_Temp < Temp_Threshold && Average_Pulse >
Pulse_Threshold)
    {
        TrackA_Count = TrackA_Count +1;
    }
    else if (Average_Temp > Temp_Threshold && Average_Pulse <
Pulse_Threshold)
    {
        TrackB_Count = TrackB_Count +1;
    }
    else if (Average_Temp < Temp_Threshold && Average_Pulse <
Pulse_Threshold)
    {
        Default_Count = Default_Count +1;
    }
    else if (Average_Temp > Temp_Threshold && Average_Pulse ==
Pulse_Threshold)

```

```

        {
            TrackB_Count = TrackB_Count +1;
        }
        else if (Average_Temp < Temp_Threshold && Average_Pulse ==
Pulse_Threshold)
        {
            TrackA_Count = TrackA_Count +1;
        }
        else if (Average_Temp == Temp_Threshold && Average_Pulse ==
Pulse_Threshold)
        {
            Default_Count = Default_Count +1;
        }
        else if (Average_Temp == Temp_Threshold && Average_Pulse <
Pulse_Threshold)
        {
            Default_Count = Default_Count +1;
        }
        else if (Average_Temp == Temp_Threshold && Average_Pulse >
Pulse_Threshold)
        {
            TrackA_Count = TrackA_Count +1;
        }
        else if (TrackA_Count == 0 && TrackB_Count == 0 && Default_Count ==
0)
        {
            Random_Count= Random_Count + 1;
        }
    }

```

```

void DetermineChoice(void)
{
    if (TrackA_Count > TrackB_Count && TrackA_Count > Default_Count &&
TrackA_Count > Random_Count)
    {
        state = 1;
    }
    else if (TrackB_Count > TrackA_Count && TrackB_Count > Default_Count
&& TrackB_Count > Random_Count)
    {
        state = 2;
    }
    else if (Default_Count > TrackA_Count && Default_Count > Random_Count
&& Default_Count > TrackB_Count)
    {
        state = 0;
    }
    else if (Random_Count > TrackA_Count && Random_Count > Default_Count
&& Random_Count > TrackB_Count)
    {
        state = 3;
    }
    else if (TrackA_Count == TrackB_Count && TrackA_Count > Default_Count
&& TrackA_Count > Random_Count)
    {
        state = 0;
    }
}

```

```

    else if (TrackA_Count == Default_Count && TrackA_Count > TrackB_Count
&& TrackA_Count > Random_Count)
    {
        state = 0;
    }
    else if (TrackA_Count == Random_Count && TrackA_Count > TrackB_Count
&& TrackA_Count > Default_Count)
    {
        state = 1;
    }
    else if (TrackB_Count == Default_Count && TrackB_Count > TrackA_Count
&& TrackB_Count > Random_Count)
    {
        state = 0;
    }
    else if (TrackB_Count == Random_Count && TrackB_Count > TrackA_Count
&& TrackB_Count > Default_Count)
    {
        state = 2;
    }
    else if (Default_Count == Random_Count && Default_Count >
TrackA_Count && Default_Count > TrackB_Count)
    {
        state = 0;
    }
}

```

```

void TrackDecision(void)
{

```

```

    switch(state)
    {
        case 0:
            //Default Track
            BSP_LED_On(LED_GREEN);
            break;

        case 1:
            //Track A
            run = 0;
            while(state == 1)
            {
                BSP_LED_On(LED_RED);
                if (run == 0)
                {
                    rotate_angle_forwards(90);
                    osDelay(10000);
                    run = 1;
                    rotate_angle_backwards(90);
                    run = 1;
                }
                EmergencyStop();
                OverrideSystem();
            }
            break;

        case 2:
            //Track B
            run = 1;

```

```

while(state == 2)
{
    BSP_LED_On(LED_YELLOW);
    if (run == 1)
    {
        rotate_angle_backwards(90);
        osDelay(10000);
        run = 0;
        rotate_angle_forwards(90);
        run = 0;
    }
    EmergencyStop();
    OverrideSystem();
}
break;

case 3:
//Random

    for(int i = 0; i < 5; i++)
    {
        BSP_LED_Off(LED_RED);
        BSP_LED_On(LED_GREEN);
        osDelay(100);
        BSP_LED_Off(LED_GREEN);
        BSP_LED_On(LED_YELLOW);
        osDelay(100);
        BSP_LED_Off(LED_YELLOW);
        BSP_LED_On(LED_RED);
        osDelay(100);
    }

    osDelay(1000);

    int r = rand() % 4;

    if (r == 1)
    {
        //track A
        BSP_LED_On(LED_RED);
        rotate_angle_forwards(90);
        osDelay(10000);
        rotate_angle_backwards(90);
        OverrideSystem();
    }
    else if (r == 2)
    {
        //Track B
        BSP_LED_On(LED_YELLOW);
        rotate_angle_backwards(90);
        osDelay(10000);
        rotate_angle_forwards(90);
        OverrideSystem();
    }
    else if (r == 3)
    {
        //Default Track
        BSP_LED_On(LED_GREEN);
    }
}

```

```

    }
    break;

case 4:
    //Emergency Stop
    BSP_LED_Off(LED_GREEN);
    BSP_LED_Off(LED_YELLOW);
    for(;;)
    {
        BSP_LED_On(LED_RED);
        osDelay(100);
        BSP_LED_Off(LED_RED);
        osDelay(100);
    }
    break;

case 5:
    //Override
    BSP_LED_On(LED_GREEN);
    BSP_LED_On(LED_YELLOW);
    BSP_LED_On(LED_RED);

    while(state == 5)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == 0)
/*Track B*/
        {
            run = 1;
            BSP_LED_Off(LED_GREEN);
            BSP_LED_Off(LED_YELLOW);
            BSP_LED_On(LED_RED);
            if (run == 1)
            {
                rotate_angle_backwards(90);
                osDelay(10000);
                run = 0;
                rotate_angle_forwards(90);
                run = 0;
            }
            osDelay(5);
        }
        else if (HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_8) == 0)
/*Track A*/
        {
            run = 0;
            BSP_LED_Off(LED_GREEN);
            BSP_LED_Off(LED_RED);
            BSP_LED_On(LED_YELLOW);
            if (run == 0)
            {
                rotate_angle_forwards(90);
                osDelay(10000);
                run = 1;
                rotate_angle_backwards(90);
                run = 1;
            }
            osDelay(5);
        }
    }
}

```

```

        }
        else if (HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_7) == 0)
/*Default*/
        {
            BSP_LED_Off(LED_RED);
            BSP_LED_Off(LED_YELLOW);
            BSP_LED_On(LED_GREEN);
        }
        else if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0)
        {
            state = 4;
        }
        osDelay(100);
    }

    osDelay(100);
    break;
}

}

/* USER CODE END 4 */

/* USER CODE BEGIN Header_StartDefaultTask */
/**
 * @brief Function implementing the defaultTask thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void const * argument)
{
    for (;;)
    {
        uint32_t start_time = HAL_GetTick();

        while ((HAL_GetTick() - start_time) < 30000)
        {
            EmergencyStop();
            OverrideSystem();

            if (state == 4 || state == 5)
            {
                TrackDecision();
            }
            else if (deciding_done == 0)
            {
                if (HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_7) == 0)
                {
                    if (emotion <= 1)
                    {
                        emotion++;
                    }
                    osDelay(500);
                }
            }
        }
    }
}

```

```

        ThermalAndPulseSensor();
        camera(emotion);
        DetermineChoice();
    }

    osDelay(10);
}

EmergencyStop();
OverrideSystem();
TrackDecision();

osDelay(100);
}
}

/* USER CODE BEGIN Header_PulseSensorTaskHandler */
/**
 * @brief Function implementing the PulseSensorTask thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_PulseSensorTaskHandler */
void PulseSensorTaskHandler(void const * argument)
{
    /* USER CODE BEGIN PulseSensorTaskHandler */
    /* Infinite loop */

    for(;;)
    {

        while (state != 4 && state != 5)
        {
            /*excited*/
            if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == 0)
            {
                chosen_pulse = rand() % (max - min + 1) + min;
                Average_Pulse = Pulse_Excited[chosen_pulse];
                osDelay(100);
            }
            /*stressed*/
            else if (HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_8) == 0)
            {
                chosen_pulse = rand() % (max - min + 1) + min;
                Average_Pulse = Pulse_Scared[chosen_pulse];
                osDelay(100);
            }
        }

    }
    /* USER CODE END PulseSensorTaskHandler */
}

/* USER CODE BEGIN Header_ThermalSensorTaskHandler */
/**
 * @brief Function implementing the ThermalSensorTa thread.
 * @param argument: Not used

```

```

* @retval None
*/
/* USER CODE END Header_ThermalSensorTaskHandler */
void ThermalSensorTaskHandler(void const * argument)
{
    /* USER CODE BEGIN ThermalSensorTaskHandler */
    /* Infinite loop */
    for(;;)
    {

        float temperature_sum = 0.0;

        for(int i = 0; i < NUM_SAMPLES; i++)
        {
            HAL_ADC_Start(&hadc2);
            HAL_ADC_PollForConversion(&hadc2, 5);
            adc_value_thermal = HAL_ADC_GetValue(&hadc2);
            voltage_thermal = adc_value_thermal *
(V_REF_THERMAL/(float)ADC_RESOLUTION_THERMAL);
            temperature = voltage_thermal / TEMP_SENSITIVITY;
            temperature = temperature - 6;
            temperature_sum = temperature_sum + temperature;
            osDelay(500);
        }

        Average_Temp = temperature_sum / NUM_SAMPLES;
        HAL_ADC_Stop(&hadc2);
        osDelay(5);

    }
    /* USER CODE END ThermalSensorTaskHandler */
}

/* MPU Configuration */

void MPU_Config(void)
{
    MPU_Region_InitTypeDef MPU_InitStruct = {0};

    /* Disables the MPU */
    HAL_MPU_Disable();

    /** Initializes and configures the Region and the memory to be
protected
    */
    MPU_InitStruct.Enable = MPU_REGION_ENABLE;
    MPU_InitStruct.Number = MPU_REGION_NUMBER0;
    MPU_InitStruct.BaseAddress = 0x0;
    MPU_InitStruct.Size = MPU_REGION_SIZE_4GB;
    MPU_InitStruct.SubRegionDisable = 0x87;
    MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
    MPU_InitStruct.AccessPermission = MPU_REGION_NO_ACCESS;
    MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_DISABLE;
    MPU_InitStruct.IsShareable = MPU_ACCESS_SHAREABLE;
    MPU_InitStruct.IsCacheable = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;

```



```

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
    /* Enables the MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);

}
/*****
Interrupts */
/**
 * @brief This function handles I2C1 event interrupt.
 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return
state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line
number
 *
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line
number,
ex: printf("Wrong parameters value: file %s on line %d\r\n", file,
line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```